

Subroutine is a sequence of program instruction that performs a specific task.
It is called by another subprogram and returns to the calling subprogram after it executes.

Procedure is a subroutine **without** a return value
Function is a subroutine **with** a return value.

Subroutine Main Instructions:

call label	ret source
▪ <i>push PC</i>	▪ <i>pop PC</i>
▪ <i>jmp label</i>	▪ <i>add SP, source</i>

Stack: Data Structure

- **push**
insert element
machine instruction: **push source**
- **pop**
retrieve element
machine instruction: **pop destination**

Stack Segment

- **SS: stack segment**
 - starting address of stack
- **ESP: stack pointer**
 - top of stack pointer

Parameter Passing

Call by Value

Only the value of the parameter is passed on the called subprogram.

Call by Reference

The address of the pointer is passed on the called subprogram. This is used when changing value of the parameter on the called subprogram.

Example 1.

C program

subprogram call:
sum(x, y);

subprogram:
void sum (int a, int b)
{
 int num;
 num = a + b;
}

Assembly program

subprogram call:
push word [x]
push word [y]
call sum

subprogram:
sum:
mov ebp, esp
sub esp, 2
mov ax, [ebp + 6]
add ax, [ebp + 4]
mov [ebp -2], ax
add esp, 2
ret 4

Example 2.

C Program

subprogram call:
sum(&num, x, y);

subprogram:
void sum(int*n, int a, int b) {
 *n = a + b;
}

Assembly Program

Subprogram call:
push num
push word [x]
push word [y]
call sum

subprogram:
sum:
mov ebp, esp
mov ax, [ebp+ 6]
add ax, [ebp+ 4]
mov ebx, [ebp+ 8]
mov [ebx], ax
ret 8

Example 3.

C Program

subprogram call:
num = sum(x, y);

subprogram:
int sum (int a, int b)
{
 return(a + b);
}

Assembly Program

Subprogram call:
sub esp, 2
push word [x]
push word [y]
call sum
pop word[num]

Subprogram:
sum:
mov ebp, esp
mov ax, [ebp + 6]
add ax, [ebp + 4]
mov [ebp + 8], ax
ret 4